

Exercise 1: Inventory Management System

Scenario

You are developing an inventory management system for a warehouse. Efficient data storage and retrieval are crucial.

1. Understand the Problem

Why are Data Structures and Algorithms Essential?

A warehouse inventory system stores and manages a large number of products. The system should support operations such as adding, updating, deleting, and searching for products efficiently. Choosing an appropriate data structure and algorithm improves the overall performance of the system, reduces execution time, and ensures scalability as the inventory size increases.

Suitable Data Structures

ArrayList

- Stores products sequentially.
- Easy to implement.
- Searching, updating, and deleting require traversing the list.
- Time Complexity:
 - Add: $O(1)$ (Average)
 - Search: $O(n)$
 - Update: $O(n)$
 - Delete: $O(n)$

HashMap (Chosen Data Structure)

- Stores products using productId as the key and Product as the value.
- Provides direct access to products using their IDs.
- Performs add, update, delete, and search operations efficiently.
- Best suited for inventory management systems.

2. Setup

A Java project named InventoryManagementSystem was created.

The project contains a code folder, which includes three separate Java files:

InventoryManagementSystem

└── code

└── Product.java

└── Inventory.java

└── InventoryManagementSystem.java

- **Product.java** – Defines the Product class with attributes such as productId, productName, quantity, and price.
- **Inventory.java** – Implements the inventory operations such as adding, updating, deleting, searching, and displaying products using a HashMap.
- **InventoryManagementSystem.java** – Contains the main() method to demonstrate and test the functionality of the inventory management system.

3. Implementation

Product.java

The Product class represents each product in the inventory.

Attributes

- productId
- productName
- quantity
- price

Inventory.java

The Inventory class stores products using a `HashMap<Integer, Product>`.

```
HashMap<Integer, Product> products = new HashMap<>();
```

The following operations are implemented:

- Add Product
- Update Product
- Delete Product
- Search Product
- Display Inventory

InventoryManagementSystem.java

The InventoryManagementSystem class contains the main() method, where Product objects are created and inventory operations are demonstrated.

4. Analysis

Time Complexity

Operation	Time Complexity
Add Product	O(1) (Average)
Update Product	O(1) (Average)
Delete Product	O(1) (Average)
Search Product	O(1) (Average)

Worst Case: O(n) due to hash collisions.

Optimization

Using a HashMap allows products to be accessed directly using their unique product IDs, making inventory operations significantly faster than sequential data structures such as ArrayList or LinkedList.

Further optimizations include:

- Making data members private and using getters and setters.
- Validating duplicate product IDs before insertion.
- Adding exception handling for invalid inputs.
- Connecting the application to a database for permanent storage.
- Using a TreeMap if sorted product records are required.

5. Conclusion

The Inventory Management System was successfully implemented using a HashMap as the primary data structure. The application is divided into three separate Java files following object-oriented programming principles. The system efficiently supports adding, updating, deleting, searching, and displaying products. Since most operations execute in O(1) average time, HashMap is an excellent choice for managing large warehouse inventories efficiently.